



Flipped class — topic briefs

Oscar Esteban

Sixteen topics, one each. You draw yours on Monday 14 September, afternoon and teach it on the date its slot shows below.

PRESENTS MONDAY 21 SEPTEMBER

- 1 Containers vs. VMs
- 2 Building containers (with Docker)
- 3 Automating image building with CI/CD
- 4 From Docker Compose to Kubernetes
- 5 K8s, MicroK8s, K3s, minikube: container orchestration
- 6 Data streams with Kafka

PRESENTS THURSDAY 24 SEPTEMBER

- 7 Packaging Kubernetes apps with Helm
- 8 Container security: what isolation does not give you
- 9 Caching and typed interfaces in Flyte
- 10 Retries and idempotence: what happens after a task fails
- 11 Slurm and Kubernetes, two ways to schedule work

PRESENTS MONDAY 28 SEPTEMBER

- 12 Inside a Parquet file
- 13 Partitioning and the small-files problem
- 14 Table formats and the lakehouse
- 15 Object stores versus file systems
- 16 DataFrames with pandas, Polars and Arrow

Containers vs. VMs

Containers and VMs both isolate execution. How do you choose between them?

YOUR PRESENTATION (15 MINUTES + 10 MIN Q&A)

This could be the structure of your talk. Nonetheless, you are free to improve or modify to ensure the topic will be more accessible.

1. Technical details of each isolation mechanism
2. Why VMs will always take longer to start?
3. The demo: three measurements, side by side
4. When do you still want a VM? (with examples)

YOUR DEMO (AIM AT 2 MIN, BUT FLEXIBLE IF YOU BLEND IT NICELY WITH THE PRESENTATION)

Run the same small job in a Docker container and in a virtual machine. Report three numbers: cold start time, memory at rest, and disk footprint.

Demo should run in about 2 minutes, and should be a demo unless a teacher allows you to bring pre-baked results.

YOUR THREE DELIVERABLES

- | | |
|---|------------------------|
| C1 Angle — one slide: the topic in your own words, the question you answer, what your demo will show | Wednesday 16 September |
| C2 Demo runs — the code, a README, one command | Friday 18 September |
| C3 Deck — complete slides, demo embedded | Sunday 20 September |

All three interim materials must be uploaded into ISC Learn by 18:00; the demo as a repository link. These deliverables are mandatory, and the teacher will provide feedback to improve your work.

READING

Kurtzer, Sochat & Bauer (2017). Singularity: scientific containers for mobility of compute [doi:10.1371/journal.pone.0177459](https://doi.org/10.1371/journal.pone.0177459)
IBM Technology. Containers vs VMs: What's the difference? www.youtube.com/watch?v=cjXl-yxqGTI

NOTES

Drawn by

Monday 14 September, afternoon

Building containers (with Docker)

When an image is too large or takes long to rebuild; how do you refactor the Dockerfile?

YOUR PRESENTATION (15 MINUTES + 10 MIN Q&A)

This could be the structure of your talk. Nonetheless, you are free to improve or modify to ensure the topic will be more accessible.

1. One instruction, one layer: what the build actually produces
2. The cache, and why moving one COPY down saves four minutes
3. Multi-stage: build in an image you then throw away
4. Tags move, digests do not
5. The demo: two numbers before, two numbers after

YOUR DEMO (AIM AT 2 MIN, BUT FLEXIBLE IF YOU BLEND IT NICELY WITH THE PRESENTATION)

Demonstrate how Dockerfiles are built, caching of layers, the order of instructions, tags, digests, and multi-stage builds. Show how to reduce image size and improve build times by reordering instructions and using multi-stage builds.

Demo should run in about 2 minutes, and should be a demo unless a teacher allows you to bring pre-baked results.

YOUR THREE DELIVERABLES

- | | |
|---|------------------------|
| C1 Angle — one slide: the topic in your own words, the question you answer, what your demo will show | Wednesday 16 September |
| C2 Demo runs — the code, a README, one command | Friday 18 September |
| C3 Deck — complete slides, demo embedded | Sunday 20 September |

All three interim materials must be uploaded into ISC Learn by 18:00; the demo as a repository link. These deliverables are mandatory, and the teacher will provide feedback to improve your work.

READING

Merkel (2014). Docker: lightweight Linux containers for consistent development and deployment

[doi:10.5555/2600239.2600241](https://doi.org/10.5555/2600239.2600241)

Better Stack. Docker Image BEST Practices - From 1.2GB to 10MB www.youtube.com/watch?v=t779DVjCKCs

TechWorld with Nana. Top 8 Docker Best Practices for using Docker in Production. www.youtube.com/watch?v=8vXoMqWgbQQ

NOTES

Drawn by

Monday 14 September, afternoon

Automating image building with CI/CD

How do you save time, and make the process repeatable, when building images and uploading them for distribution?

YOUR PRESENTATION (15 MINUTES + 10 MIN Q&A)

This could be the structure of your talk. Nonetheless, you are free to improve or modify to ensure the topic will be more accessible.

1. Doing every step manually: build, push and see.
2. GitHub Actions: anatomy of a workflow file
3. GHAs: how workflows are triggered, show them in action
4. Building docker images within GHA
5. Pushing a commit and see the action kick in and build-push a new image.

YOUR DEMO (AIM AT 2 MIN, BUT FLEXIBLE IF YOU BLEND IT NICELY WITH THE PRESENTATION)

A repository whose workflow builds and publishes the image on every push. Show a commit triggering it, the image appearing in the registry, and a second commit that reuses the cache.

Demo should run in about 2 minutes, and should be a demo unless a teacher allows you to bring pre-baked results.

YOUR THREE DELIVERABLES

- | | |
|---|------------------------|
| C1 Angle — one slide: the topic in your own words, the question you answer, what your demo will show | Wednesday 16 September |
| C2 Demo runs — the code, a README, one command | Friday 18 September |
| C3 Deck — complete slides, demo embedded | Sunday 20 September |

All three interim materials must be uploaded into ISC Learn by 18:00; the demo as a repository link. These deliverables are mandatory, and the teacher will provide feedback to improve your work.

READING

TechWorld with Nana. GitHub Actions Tutorial - Basic Concepts and CI/CD Pipeline with Docker www.youtube.com/watch?v=R8_veQiYBjI

NOTES

Drawn by

Monday 14 September, afternoon

From Docker Compose to Kubernetes

Compose already runs several containers that find each other. So what does Kubernetes add?

YOUR PRESENTATION (15 MINUTES + 10 MIN Q&A)

This could be the structure of your talk. Nonetheless, you are free to improve or modify to ensure the topic will be more accessible.

1. Two containers, one network: how Compose makes them find each other
2. The same two services, on Kubernetes
3. The demo: same app, both ways, side by side
4. What Kubernetes buys, and what it charges for it

YOUR DEMO (AIM AT 2 MIN, BUT FLEXIBLE IF YOU BLEND IT NICELY WITH THE PRESENTATION)

Start from a two-service Compose file and show the containers reaching each other by name. Then run the same two services on a local Kubernetes and show what replaced the Compose network — and what you had to write that Compose gave you for free.

Demo should run in about 2 minutes, and should be a demo unless a teacher allows you to bring pre-baked results.

YOUR THREE DELIVERABLES

- | | |
|---|------------------------|
| C1 Angle — one slide: the topic in your own words, the question you answer, what your demo will show | Wednesday 16 September |
| C2 Demo runs — the code, a README, one command | Friday 18 September |
| C3 Deck — complete slides, demo embedded | Sunday 20 September |

All three interim materials must be uploaded into ISC Learn by 18:00; the demo as a repository link. These deliverables are mandatory, and the teacher will provide feedback to improve your work.

READING

IBM Technology. Container Orchestration Explained www.youtube.com/watch?v=kBF6Bvth0zw

IBM Technology. Kubernetes vs. Docker: It's Not an Either/Or Question www.youtube.com/watch?v=2vMEQ5zs1ko

NOTES

Drawn by

Monday 14 September, afternoon

K8s, MicroK8s, K3s, minikube: container orchestration

Four things call themselves Kubernetes. What is actually different, and which one is on your laptop?

YOUR PRESENTATION (15 MINUTES + 10 MIN Q&A)

This could be the structure of your talk. Nonetheless, you are free to improve or modify to ensure the topic will be more accessible.

1. One API, several distributions: what upstream Kubernetes is, in parts
2. What each distribution keeps, drops or replaces
3. The demo: the same Deployment on two of them
4. Which one to reach for, and why this course runs k3s

YOUR DEMO (AIM AT 2 MIN, BUT FLEXIBLE IF YOU BLEND IT NICELY WITH THE PRESENTATION)

Bring the same Deployment up on at least two of them. Show what differs where it is visible: install time, what is running on the node, what is missing, and what happens when you kill a pod.

Demo should run in about 2 minutes, and should be a demo unless a teacher allows you to bring pre-baked results.

YOUR THREE DELIVERABLES

- | | |
|---|------------------------|
| C1 Angle — one slide: the topic in your own words, the question you answer, what your demo will show | Wednesday 16 September |
| C2 Demo runs — the code, a README, one command | Friday 18 September |
| C3 Deck — complete slides, demo embedded | Sunday 20 September |

All three interim materials must be uploaded into ISC Learn by 18:00; the demo as a repository link. These deliverables are mandatory, and the teacher will provide feedback to improve your work.

READING

Burns et al. (2016). Borg, Omega, and Kubernetes [doi:10.1145/2890784](https://doi.org/10.1145/2890784).
TechWorld with Nana. Minikube and Kubectl explained | Setup for Beginners | Kubernetes Tutorial 17 www.youtube.com/watch?v=E2pP1MOfo3g
MicroK8s vs K3s vs minikube canonical.com/microk8s/compare
Zakhar Snezhkin. Small Kubernetes for your local experiments: k0s, MicroK8s, kind, k3s, and Minikube palark.com/blog/small-local-kubernetes-comparison/

NOTES

Drawn by

Monday 14 September, afternoon

Data streams with Kafka

How do you handle data that is governed by events (as opposed to regular sampling)?

YOUR PRESENTATION (15 MINUTES + 10 MIN Q&A)

This could be the structure of your talk. Nonetheless, you are free to improve or modify to ensure the topic will be more accessible.

1. A queue forgets, a log does not: append, offset, retention
2. Topics and partitions, and what ordering you actually get
3. The demo: kill the broker, bring it back, read it anyway
4. Two consumers, one topic, and who tracks how far each has read

YOUR DEMO (AIM AT 2 MIN, BUT FLEXIBLE IF YOU BLEND IT NICELY WITH THE PRESENTATION)

Run a single-broker Kafka from a compose file. Produce a handful of messages to a topic, kill the broker, bring it back, and consume from offset 0 to show nothing was lost. Then attach a second consumer in its own group and show it reads the messages the first one has already read.

Demo should run in about 2 minutes, and should be a demo unless a teacher allows you to bring pre-baked results.

YOUR THREE DELIVERABLES

- | | |
|---|------------------------|
| C1 Angle — one slide: the topic in your own words, the question you answer, what your demo will show | Wednesday 16 September |
| C2 Demo runs — the code, a README, one command | Friday 18 September |
| C3 Deck — complete slides, demo embedded | Sunday 20 September |

All three interim materials must be uploaded into ISC Learn by 18:00; the demo as a repository link. These deliverables are mandatory, and the teacher will provide feedback to improve your work.

READING

Kreps, Narkhede & Rao (2011). Kafka: a distributed messaging system for log processing cwiki.apache.org/confluence/download/attachments/27822226/Kafka-netdb-06-2011.pdf

IBM Technology. What is Kafka? www.youtube.com/watch?v=aj9CDZm0Glc

Stephane Maarek. How do Kafka Consumer Groups and Consumer Offsets work in Apache Kafka? www.youtube.com/watch?v=9o5LAbPnc28

NOTES

Drawn by

Monday 14 September, afternoon

Packaging Kubernetes apps with Helm

How do you ship the same application to a test cluster and a production cluster without maintaining two copies of its manifests?

YOUR PRESENTATION (15 MINUTES + 10 MIN Q&A)

This could be the structure of your talk. Nonetheless, you are free to improve or modify to ensure the topic will be more accessible.

1. The problem: the same app, several clusters, one set of manifests
2. What a chart is: templates, values, and the release as a unit
3. The demo: one chart, two values files, two deployments
4. Where templating YAML breaks down, and what people reach for instead

YOUR DEMO (AIM AT 2 MIN, BUT FLEXIBLE IF YOU BLEND IT NICELY WITH THE PRESENTATION)

Install a public chart in one command and watch a working application come up. Then take that same chart with two different values files and show two genuinely different deployments coming out of one package.

Demo should run in about 2 minutes, and should be a demo unless a teacher allows you to bring pre-baked results.

YOUR THREE DELIVERABLES

- | | |
|---|------------------------|
| C1 Angle — one slide: the topic in your own words, the question you answer, what your demo will show | Saturday 19 September |
| C2 Demo runs — the code, a README, one command | Monday 21 September |
| C3 Deck — complete slides, demo embedded | Wednesday 23 September |

All three interim materials must be uploaded into ISC Learn by 18:00; the demo as a repository link. These deliverables are mandatory, and the teacher will provide feedback to improve your work.

READING

Iglesias (2016). Helm Charts: making it simple to package and deploy common applications on Kubernetes kubernetes.io/blog/2016/10/helm-charts-making-it-simple-to-package-and-deploy-apps-on-kubernetes/
Helm documentation. Charts helm.sh/docs/topics/charts/
TechWorld with Nana. What is Helm in Kubernetes? Helm and Helm Charts explained | Kubernetes Tutorial 23 www.youtube.com/watch?v=ykwb1d0DXU

NOTES

Drawn by

Monday 14 September, afternoon

Container security: what isolation does not give you

Your container runs as root. What can it reach on the host, and what stops it?

YOUR PRESENTATION (15 MINUTES + 10 MIN Q&A)

This could be the structure of your talk. Nonetheless, you are free to improve or modify to ensure the topic will be more accessible.

1. Root in the container is root on the host, unless you say otherwise
2. What the kernel actually separates: namespaces, cgroups, capabilities
3. The demo: the same write, before and after dropping privilege
4. Why `--privileged` shows up in tutorials, and what it costs

YOUR DEMO (AIM AT 2 MIN, BUT FLEXIBLE IF YOU BLEND IT NICELY WITH THE PRESENTATION)

Bind-mount a host directory into a container, write a file from inside as root, and show it on the host owned by root. Then run the same image as a non-root user with capabilities dropped, and show the write fail. Keep it to what you can demonstrate on your own laptop.

Demo should run in about 2 minutes, and should be a demo unless a teacher allows you to bring pre-baked results.

YOUR THREE DELIVERABLES

- | | |
|---|------------------------|
| C1 Angle — one slide: the topic in your own words, the question you answer, what your demo will show | Saturday 19 September |
| C2 Demo runs — the code, a README, one command | Monday 21 September |
| C3 Deck — complete slides, demo embedded | Wednesday 23 September |

All three interim materials must be uploaded into ISC Learn by 18:00; the demo as a repository link. These deliverables are mandatory, and the teacher will provide feedback to improve your work.

READING

Combe, Martin & Di Pietro (2016). To Docker or Not to Docker: A Security Perspective [doi:10.1109/MCC.2016.100](https://doi.org/10.1109/MCC.2016.100)
 IBM Technology. Container Security Explained www.youtube.com/watch?v=b_euX_M82ul
 Container Security Fundamentals: Linux Capabilities (Part 1) www.youtube.com/watch?v=IBlvPys7L8w

NOTES

Drawn by

Monday 14 September, afternoon

Caching and typed interfaces in Flyte

You changed one task and re-ran the workflow. What has to run again, and who decides?

YOUR PRESENTATION (15 MINUTES + 10 MIN Q&A)

This could be the structure of your talk. Nonetheless, you are free to improve or modify to ensure the topic will be more accessible.

1. The problem: re-running work you have already done
2. What the cache keys on, and how you invalidate it when it lies to you
3. Typed interfaces: the failure that arrives before you spend the compute
4. The demo: run, re-run, change one input, break a type

YOUR DEMO (AIM AT 2 MIN, BUT FLEXIBLE IF YOU BLEND IT NICELY WITH THE PRESENTATION)

Run a Flyte workflow. Run it again unchanged and show the cache hits. Change one task's input and show exactly what re-runs and what does not. Then break a type on purpose and show the failure arrive before anything executes.

Demo should run in about 2 minutes, and should be a demo unless a teacher allows you to bring pre-baked results.

YOUR THREE DELIVERABLES

- | | |
|---|------------------------|
| C1 Angle — one slide: the topic in your own words, the question you answer, what your demo will show | Saturday 19 September |
| C2 Demo runs — the code, a README, one command | Monday 21 September |
| C3 Deck — complete slides, demo embedded | Wednesday 23 September |

All three interim materials must be uploaded into ISC Learn by 18:00; the demo as a repository link. These deliverables are mandatory, and the teacher will provide feedback to improve your work.

READING

Flyte documentation. Caching docs.flyte.org/en/latest/user_guide/development_lifecycle/caching.html
Samhita Alla. Run Your Data and Machine Learning Workflows on Kubernetes with Flyte www.youtube.com/watch?v=nXZHtaTnp14

NOTES

Drawn by

Monday 14 September, afternoon

Retries and idempotence: what happens after a task fails

How to use retry to build more dependable pipelines? When are tasks really retryable?

YOUR PRESENTATION (15 MINUTES + 10 MIN Q&A)

This could be the structure of your talk. Nonetheless, you are free to improve or modify to ensure the topic will be more accessible.

1. Retries, and the backoff that stops them turning one failure into many
2. A task is retryable when running it twice leaves the world as running it once did
3. The ones that pass that test, and the ones that only look like they do
4. The demo, and what has to be in the logs to tell those two apart

YOUR DEMO (AIM AT 2 MIN, BUT FLEXIBLE IF YOU BLEND IT NICELY WITH THE PRESENTATION)

A Flyte task that fails at random. Add a retry policy and show it recover. Then make it non-idempotent, appending to a file, and show the retry produce a quietly wrong answer. Fix it, and show the difference in the logs.

Demo should run in about 2 minutes, and should be a demo unless a teacher allows you to bring pre-baked results.

YOUR THREE DELIVERABLES

- | | |
|---|------------------------|
| C1 Angle — one slide: the topic in your own words, the question you answer, what your demo will show | Saturday 19 September |
| C2 Demo runs — the code, a README, one command | Monday 21 September |
| C3 Deck — complete slides, demo embedded | Wednesday 23 September |

All three interim materials must be uploaded into ISC Learn by 18:00; the demo as a repository link. These deliverables are mandatory, and the teacher will provide feedback to improve your work.

READING

Helland (2012). Idempotence Is Not a Medical Condition [doi:10.1145/2181796.2187821](https://doi.org/10.1145/2181796.2187821)
 Beyer et al. Site Reliability Engineering, ch. 22: Addressing Cascading Failures sre.google/sre-book/addressing-cascading-failures/

NOTES

Drawn by

Monday 14 September, afternoon

Slurm and Kubernetes, two ways to schedule work

Your job will run on a machine that is not yours. What does Slurm need to know before it starts, and what does Kubernetes want instead?

YOUR PRESENTATION (15 MINUTES + 10 MIN Q&A)

This could be the structure of your talk. Nonetheless, you are free to improve or modify to ensure the topic will be more accessible.

1. Both tools say scheduling and they do not mean the same thing by it
2. Slurm gives you an allocation of machines for a while and runs your script on it
3. Kubernetes takes a description of what should be true and works to keep it true
4. The demo, the same job both ways, and which model fits which work

YOUR DEMO (AIM AT 2 MIN, BUT FLEXIBLE IF YOU BLEND IT NICELY WITH THE PRESENTATION)

Submit the same small job twice, once as a Slurm batch script and once as a Kubernetes Job. Show what each makes you declare up front, how each queues you, and what you get back when it is done.

Demo should run in about 2 minutes, and should be a demo unless a teacher allows you to bring pre-baked results.

YOUR THREE DELIVERABLES

- | | |
|---|------------------------|
| C1 Angle — one slide: the topic in your own words, the question you answer, what your demo will show | Saturday 19 September |
| C2 Demo runs — the code, a README, one command | Monday 21 September |
| C3 Deck — complete slides, demo embedded | Wednesday 23 September |

All three interim materials must be uploaded into ISC Learn by 18:00; the demo as a repository link. These deliverables are mandatory, and the teacher will provide feedback to improve your work.

READING

Yoo, Jette & Grondona (2003). SLURM: Simple Linux Utility for Resource Management [doi:10.1007/10968987_3](https://doi.org/10.1007/10968987_3)
Beltre et al. (2019). Enabling HPC Workloads on Cloud Infrastructure Using Kubernetes Container Orchestration Mechanisms [doi:10.1109/CANOPIE-HPC49598.2019.00007](https://doi.org/10.1109/CANOPIE-HPC49598.2019.00007)
NAISS. Lecture 3: Introduction to the batch scheduler Slurm www.youtube.com/watch?v=GA3xZ-eG6yk

NOTES

Inside a Parquet file

Why is the same data both smaller and faster to query as Parquet than as CSV?

YOUR PRESENTATION (15 MINUTES + 10 MIN Q&A)

This could be the structure of your talk. Nonetheless, you are free to improve or modify to ensure the topic will be more accessible.

1. A CSV stores a row at a time, Parquet stores a column at a time
2. Why a column of one type compresses when a row of mixed types does not
3. Row groups, column chunks, and the footer that lets a reader skip most of the file
4. The demo, size first and then the time to read one column

YOUR DEMO (AIM AT 2 MIN, BUT FLEXIBLE IF YOU BLEND IT NICELY WITH THE PRESENTATION)

Write one dataset as CSV and as Parquet, and compare size. Then read a single column from each and compare time and bytes actually read. Finish by showing the row-group and column-chunk structure of the file itself.

Demo should run in about 2 minutes, and should be a demo unless a teacher allows you to bring pre-baked results.

YOUR THREE DELIVERABLES

- | | |
|---|------------------------|
| C1 Angle — one slide: the topic in your own words, the question you answer, what your demo will show | Wednesday 23 September |
| C2 Demo runs — the code, a README, one command | Friday 25 September |
| C3 Deck — complete slides, demo embedded | Sunday 27 September |

All three interim materials must be uploaded into ISC Learn by 18:00; the demo as a repository link. These deliverables are mandatory, and the teacher will provide feedback to improve your work.

READING

Melnik et al. (2010). Dremel: interactive analysis of web-scale datasets [doi:10.14778/1920841.1920886](https://doi.org/10.14778/1920841.1920886)
 Abadi, Boncz, Harizopoulos, Idreos & Madden (2013). The Design and Implementation of Modern Column-Oriented Database Systems [doi:10.1561/19000000024](https://doi.org/10.1561/19000000024)
 Julien Le Dem. The columnar roadmap: Apache Parquet and Apache Arrow www.youtube.com/watch?v=dPb2ZXnt2_U
 Row Groups in Apache Parquet www.youtube.com/watch?v=BjssGRISnfA

NOTES

Drawn by

Monday 14 September, afternoon

Partitioning and the small-files problem

You partitioned the dataset by day and the same query got slower. Where did the time go?

YOUR PRESENTATION (15 MINUTES + 10 MIN Q&A)

This could be the structure of your talk. Nonetheless, you are free to improve or modify to ensure the topic will be more accessible.

1. Partitioning a dataset by making a directory for each value in a column (for example: "date")
2. Why that partition only works if analysis queries only for single days
3. If instead, you'd typically run 1000-day queries, then access cost raises
4. The demo uses the MeteoSuisse dataset to demonstrate the performance of two distant layouts

YOUR DEMO (AIM AT 2 MIN, BUT FLEXIBLE IF YOU BLEND IT NICELY WITH THE PRESENTATION)

Take the MeteoSuisse measurements and write them twice, once with a directory per day and once with a directory per year. Run a single-day query against both, then a query spanning three years against both. Report the read time and the number of files opened in each of the four cases.

Demo should run in about 2 minutes, and should be a demo unless a teacher allows you to bring pre-baked results.

YOUR THREE DELIVERABLES

- | | |
|---|------------------------|
| C1 Angle — one slide: the topic in your own words, the question you answer, what your demo will show | Wednesday 23 September |
| C2 Demo runs — the code, a README, one command | Friday 25 September |
| C3 Deck — complete slides, demo embedded | Sunday 27 September |

All three interim materials must be uploaded into ISC Learn by 18:00; the demo as a repository link. These deliverables are mandatory, and the teacher will provide feedback to improve your work.

READING

Armbrust et al. (2020). Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores

[doi:10.14778/3415478.3415560](https://doi.org/10.14778/3415478.3415560)

Matthew Powers. Optimizing Delta Parquet Data Lakes for Apache Spark www.youtube.com/watch?v=euBvYFTVN8s

NOTES

Drawn by

Monday 14 September, afternoon

Table formats and the lakehouse

A lake is a directory of files. What does it take to call it a table?

YOUR PRESENTATION (15 MINUTES + 10 MIN Q&A)

This could be the structure of your talk. Nonetheless, you are free to improve or modify to ensure the topic will be more accessible.

1. Without a table format, a reader finds the data by listing the directory and reading every file it finds
2. If a writer is adding files while that listing happens, the reader gets some of them and not others, and its answer mixes before and after
3. Iceberg keeps a metadata file listing the data files in each snapshot, so a reader stays on the snapshot it opened while the writer builds the next
4. The demo does that same append twice, once on a plain folder and once on an Iceberg table, then adds a column and re-runs a query written before the change

YOUR DEMO (AIM AT 2 MIN, BUT FLEXIBLE IF YOU BLEND IT NICELY WITH THE PRESENTATION)

Write Parquet, then append to it while a reader is running, and show what the reader sees. Do the same through an Iceberg table and show the reader get a consistent snapshot. Finish by changing the schema and showing old readers survive it.

Demo should run in about 2 minutes, and should be a demo unless a teacher allows you to bring pre-baked results.

YOUR THREE DELIVERABLES

- | | |
|---|------------------------|
| C1 Angle — one slide: the topic in your own words, the question you answer, what your demo will show | Wednesday 23 September |
| C2 Demo runs — the code, a README, one command | Friday 25 September |
| C3 Deck — complete slides, demo embedded | Sunday 27 September |

All three interim materials must be uploaded into ISC Learn by 18:00; the demo as a repository link. These deliverables are mandatory, and the teacher will provide feedback to improve your work.

READING

Armbrust et al. (2020). Delta Lake: high-performance ACID table storage over cloud object stores

[doi:10.14778/3415478.3415560](https://doi.org/10.14778/3415478.3415560)

Armbrust, Ghodsi, Xin & Zaharia (2021). Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics www.cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf

Ryan Blue. Apache Iceberg: A Table Format for Huge Analytic Datasets www.youtube.com/watch?v=mf8Hb0col6o

NOTES

Drawn by

Monday 14 September, afternoon

Object stores versus file systems

What is the difference between a filesystem and an object store?

YOUR PRESENTATION (15 MINUTES + 10 MIN Q&A)

This could be the structure of your talk. Nonetheless, you are free to improve or modify to ensure the topic will be more accessible.

1. A filesystem gives you a tree of directories and files you can open, seek into, append to and rename
2. An object store gives you a flat set of keys, each holding bytes you write whole and read back whole or by byte range
3. On an object store a rename means copy and delete, listing a prefix is a paged API call, and reading part of a file needs a range request
4. The demo times those three operations on MinIO and on a local path and says which side pays for each

YOUR DEMO (AIM AT 2 MIN, BUT FLEXIBLE IF YOU BLEND IT NICELY WITH THE PRESENTATION)

Run the same operations against a local object store and against a filesystem path. Rename a directory, list a prefix with many keys, read the middle of a large file. Report which operations are cheap on each and which are quietly expensive. MinIO is enough for this. Its community edition was archived in April 2026, so existing builds still run but get no further releases, and new users are pointed at AIStor Free, which is free and single-node. Both speak the same S3 API, so either one works here.

Demo should run in about 2 minutes, and should be a demo unless a teacher allows you to bring pre-baked results.

YOUR THREE DELIVERABLES

- | | |
|---|------------------------|
| C1 Angle — one slide: the topic in your own words, the question you answer, what your demo will show | Wednesday 23 September |
| C2 Demo runs — the code, a README, one command | Friday 25 September |
| C3 Deck — complete slides, demo embedded | Sunday 27 September |

All three interim materials must be uploaded into ISC Learn by 18:00; the demo as a repository link. These deliverables are mandatory, and the teacher will provide feedback to improve your work.

READING

Ghemawat, Gobioff & Leung (2003). The Google File System [doi:10.1145/945445.945450](https://doi.org/10.1145/945445.945450)
 Brantner, Florescu, Graf, Kossmann & Kraska (2008). Building a Database on S3 [doi:10.1145/1376616.1376645](https://doi.org/10.1145/1376616.1376645)
 IBM Technology. What is Object Storage? www.youtube.com/watch?v=ZfTOQJLsAs
 MinIO for Developers, Session 1: Understanding Object Storage, Buckets, and S3 www.youtube.com/watch?v=gY090GEDdu8

NOTES

Drawn by

Monday 14 September, afternoon

DataFrames with pandas, Polars and Arrow

pandas and Polars both give you a DataFrame. What is Arrow then?

YOUR PRESENTATION (15 MINUTES + 10 MIN Q&A)

This could be the structure of your talk. Nonetheless, you are free to improve or modify to ensure the topic will be more accessible.

1. The MeteoSuisse measurements are a table of typed columns, and a DataFrame is that table plus the operations to filter, join and aggregate it
2. pandas runs each step as you wrote it, so filtering to one station after joining the station metadata reads every station first
3. Polars can read the whole chain before running it and push that filter down into the read, so most of the file never loads
4. Arrow is not a third DataFrame, it is the memory layout under both, so the same MeteoSuisse table passes from one to the other without a copy
5. The demo runs one MeteoSuisse aggregation in each, times it honestly, then hands the table across through Arrow

YOUR DEMO (AIM AT 2 MIN, BUT FLEXIBLE IF YOU BLEND IT NICELY WITH THE PRESENTATION)

The same small transformation in pandas and in Polars. Show the eager versus lazy difference, with one honest measurement. Then show the two handing data to each other through Arrow without copying it.

Demo should run in about 2 minutes, and should be a demo unless a teacher allows you to bring pre-baked results.

YOUR THREE DELIVERABLES

- | | |
|---|------------------------|
| C1 Angle — one slide: the topic in your own words, the question you answer, what your demo will show | Wednesday 23 September |
| C2 Demo runs — the code, a README, one command | Friday 25 September |
| C3 Deck — complete slides, demo embedded | Sunday 27 September |

All three interim materials must be uploaded into ISC Learn by 18:00; the demo as a repository link. These deliverables are mandatory, and the teacher will provide feedback to improve your work.

READING

McKinney (2010). Data structures for statistical computing in Python [doi:10.25080/Majora-92bf1922-00a](https://doi.org/10.25080/Majora-92bf1922-00a)
 Ritchie Vink. Polars: DataFrames in the multi-core era (PyData NYC 2023) www.youtube.com/watch?v=tqcudsykOGc
 A deep dive into the Arrow Columnar format with pyarrow and nanoarrow www.youtube.com/watch?v=k0mkDXWfLb4

NOTES